॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

CS101C: Introduction to Programming (Using C)

Instructors: Prof. Aikta Arya and Prof. Konjengbam Anand

CSE Department

Indian Institute of Technology, Dharwad

Week 5

Today's Class (31/8/2026)

Question!!

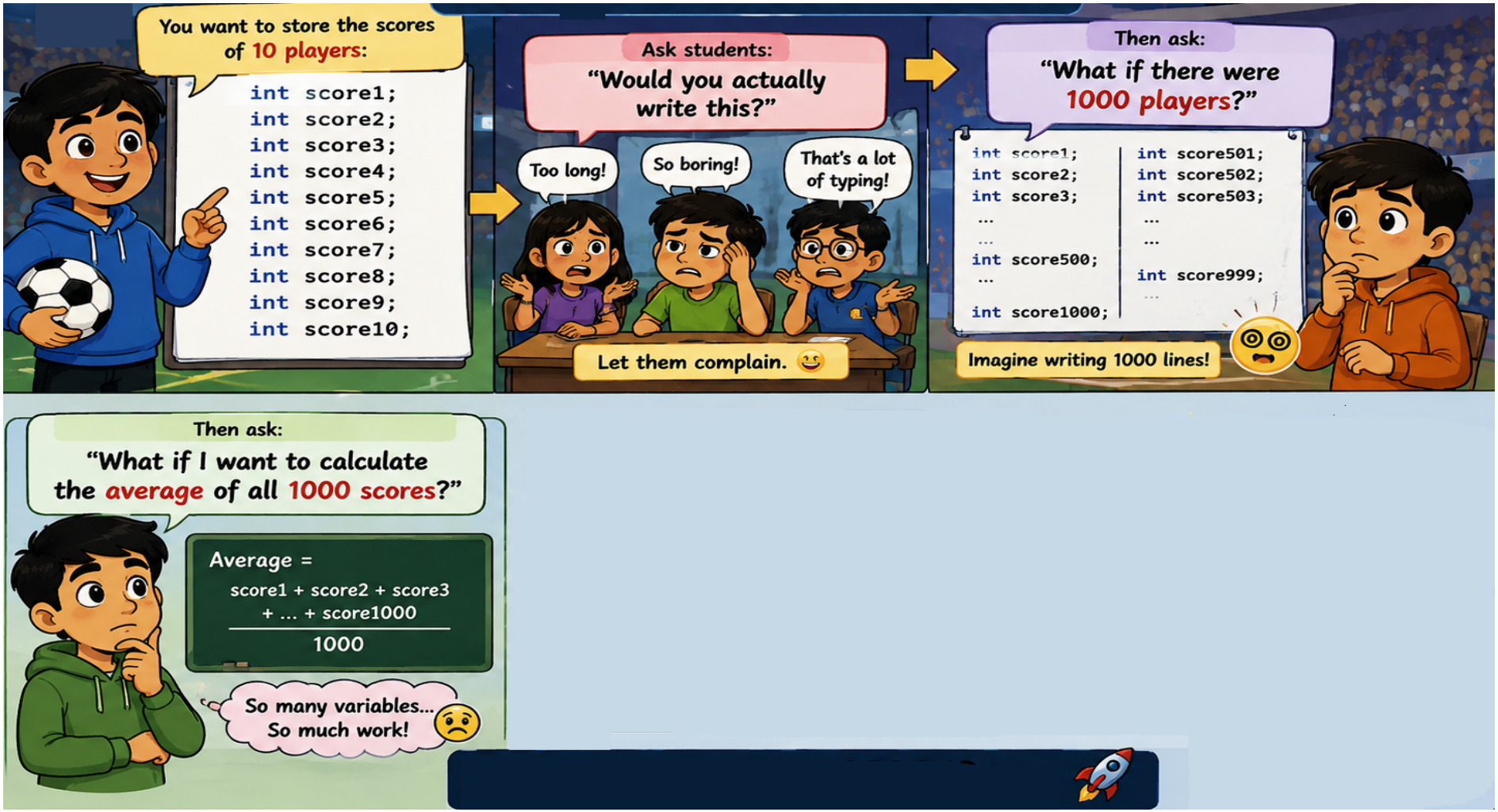
॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

- Write a C program that takes scores of 10 students as input and returns average of the 10 students.

Question!!

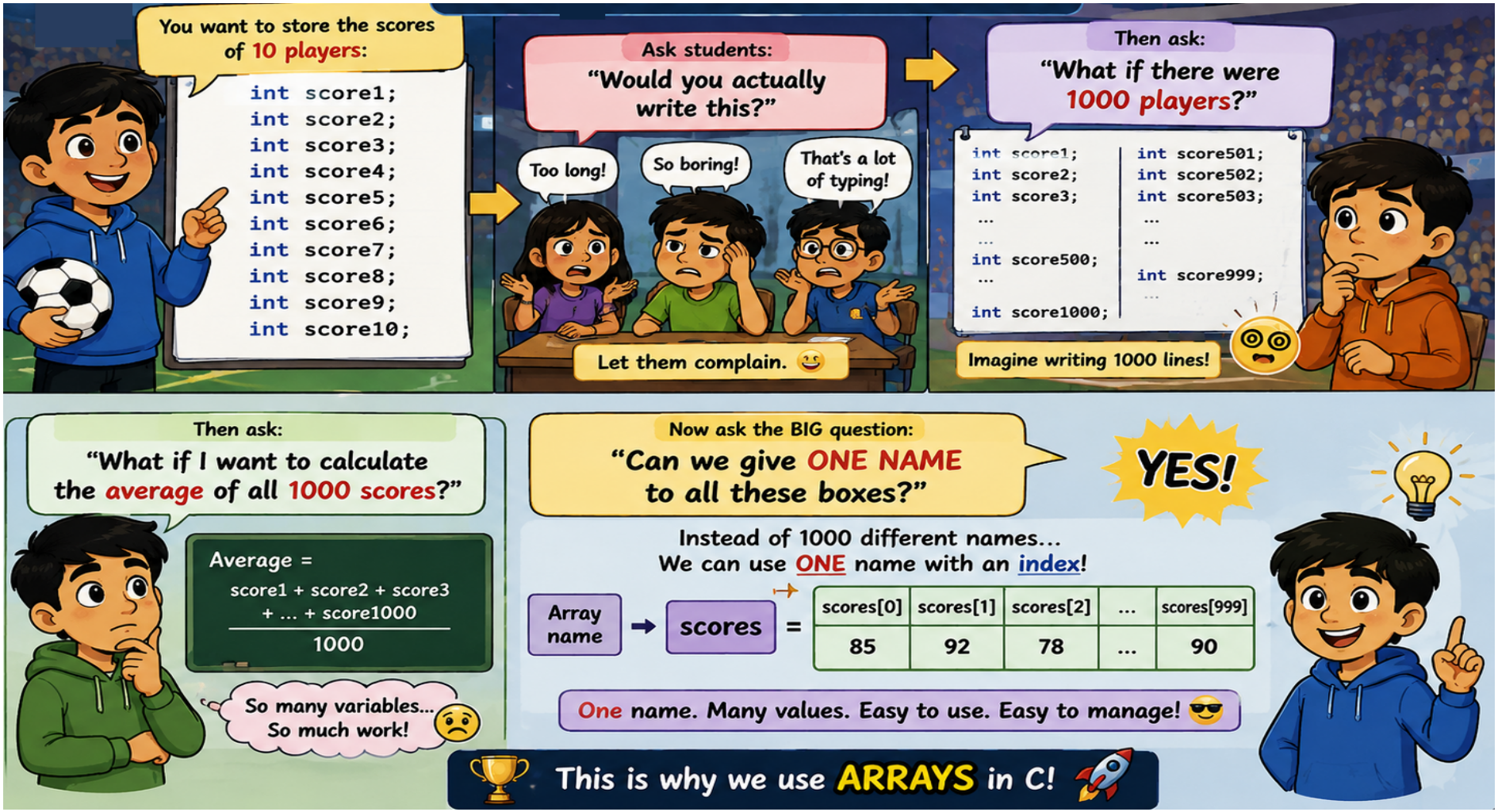
॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

- Write a C program that takes scores of 10 students as input and returns average of the 10 students.



Question!!

- Write a C program that takes scores of 10 students as input and returns average of the 10 students.



What is an Array?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

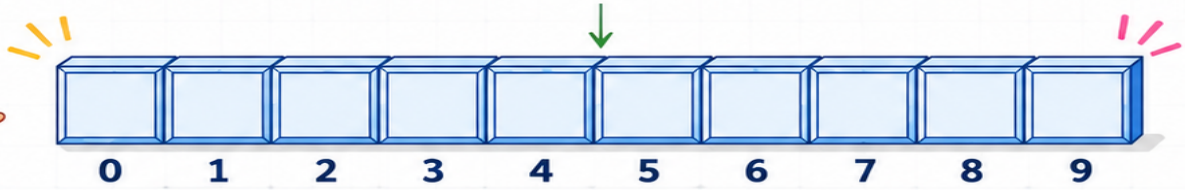
An array allows us to store multiple elements of the same type using a single name.




```
int score[10];
```

means:

score



- All elements are of type `int`
- Total 10 elements (0 to 9)
- Access using index in [ ]



How many boxes? 10

How many names? 1

All 10 boxes are represented by the single name `score`



Formal Terminology

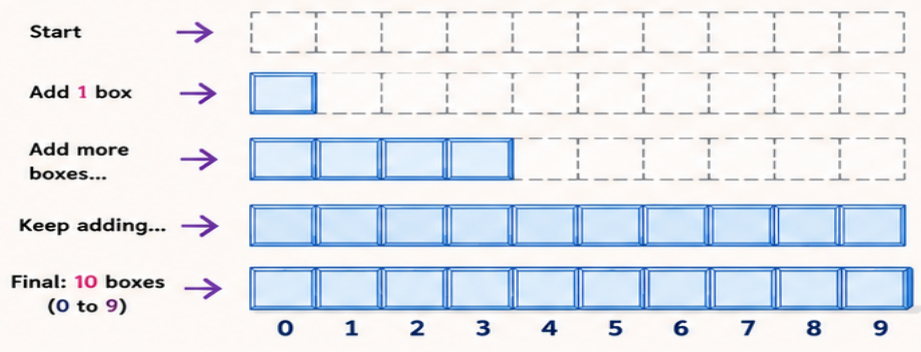
```
int score[10];
```

TYPE
Type of elements stored in the array
(here: `int`)

ARRAY NAME
Name used to refer to the whole array
(here: `score`)

SIZE
Number of elements the array can hold
(here: 10)

Animation: Creating the Array



Example: Average of 10 Students

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

```
#include <stdio.h>
int main()
{    int score[10];
     int i, sum = 0;
     float average;
     for (i = 0; i < 10; i++)
     {
          printf("Enter score %d: ", i + 1);
          scanf("%d", &score[i]);
     }
     for (i = 0; i < 10; i++)
     {
          sum = sum + score[i];
     }
     average = sum / 10.0;
     printf("Average = %.2f", average);
     return 0;
     }
```

What Does `int arr[10]` Mean?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

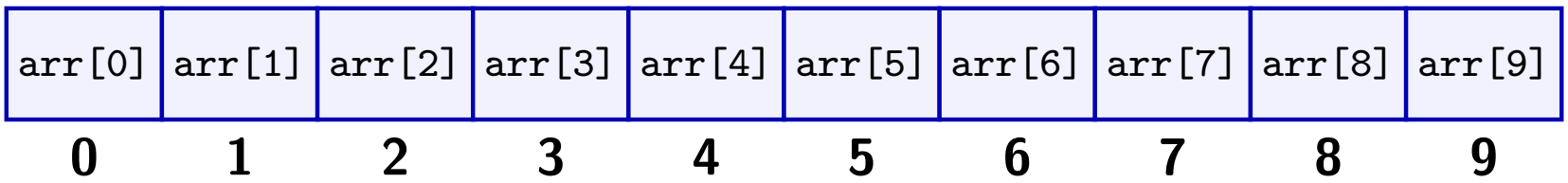
```
int arr[10];
```

`int` → **Type**

`arr` → **Array Name**

`[10]` → **Size**

`int arr[10];` **reserves space for 10 integers.**



"Where is arr[10]?" Is it the 11th box?

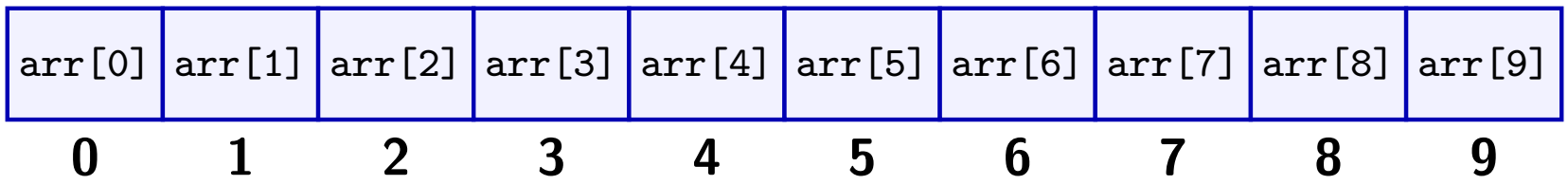
What Does `int arr[10]` Mean?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

```
int arr[10];
```

`int`	`arr`	`[10]`
↓	↓	↓
Type	**Array Name**	**Size**

`int arr[10];` **reserves space for 10 integers.**



"Where is arr[10]?" Is it the 11th box?

`arr[10]` **does NOT belong to this array!**

Because:

10 boxes $\Longrightarrow$ **indices** 0 **through** 9

Writing into an Array

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

We can store a value in any individual box:

```
arr[5] = 50;     arr[9] = 90;     arr[3] = 30;
```

Index	0	1	2	3	4	5	6	7	8	9
Value	?	?	?	30	?	50	?	?	?	90

Did we change the array name?

No!

Key Idea

What identifies the individual box?

The index / subscript

Reading from an Array

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Now let's take a value out of the array.

```
printf("%d", arr[3]);
```

Reading from an Array

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Now let's take a value out of the array.

```
printf("%d", arr[3]);
```

Think!

What will be printed?

30

Reading from an Array

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Now let's take a value out of the array.

```
printf("%d", arr[3]);
```

Think!

What will be printed?

30

```
int x = arr[5];
```

Think!

What is stored in `x`?

50

Important!

`arr[5]` **is ONE integer,**

not the whole array.

Predict the Output


```
int a[6];
a[0] = 12;
a[1] = 25;
a[2] = 7;
a[3] = 40;
a[4] = 18;
a[5] = 30;
printf("%d\n", a[2]);
printf("%d\n", a[5]);
```

Predict the Output

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

```
int a[6];
a[0] = 12;
a[1] = 25;
a[2] = 7;
a[3] = 40;
a[4] = 18;
a[5] = 30;
printf("%d\n", a[2]);
printf("%d\n", a[5]);
```

Output?

7
30

Now think about:
What would `a[6]` print?

Index	Value
0	12
1	25
2	7
3	40
4	18
5	30

Remember

6 elements $\Rightarrow$ indices **0 to 5**

`a[0]` → `a[5]` **VALID**

`a[6]` **OUT OF BOUNDS!**

Array Initialization

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Initialize an Array

```
int arr[10] = {
    10,20,30,40,50,
    60,70,80,90
};
```

Think!

We created 10 boxes but supplied how many values?

9 values!

What happens to the remaining box?

Index	0	1	2	3	4
Value	10	20	30	40	50
Index	5	6	7	8	9
Value	60	70	80	90	0

Key Rule

When an array is initialized with an initializer list, unspecified elements are initialized to **zero**.

`arr[9] = 0`

Trap Question

Are all the elements zero?

```
int arr[10];
```

What is inside the array?

Trap Question

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

Are all the elements zero?

```
int arr[10];
```

What is inside the array?

NOT necessarily 0!

Local Automatic Array

```
int arr[10];
```

The elements are **uninitialized** and contain **indeterminate values**.

Partial Initialization

```
int arr[10] = {10,20,30};
```

Remaining elements are initialized to **zero**.

Remember the Difference

`int arr[10];` → **Uninitialized**

`int arr[10] = {10,20,30};` → **Remaining elements = 0**

Question!!

- Write a C program to store 5 numbers in an array. Ask the user for a number to search, and check whether that number is present in the array.
  - If the number is found, print **"Found"**.
  - Otherwise, print **"Not Found"**.
- **Example:** Suppose our array contains: 10 25 7 40 18
  And we want to search for: 40
- **Output**: **Found**

Linear Search!!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

```
#include <stdio.h>
int main()
{    int a[5] = {10, 25, 7, 40, 18};
     int key, i, found = 0;
     printf("Enter number to search: ");
     scanf("%d", &key);
     for (i = 0; i < 5; i++)
     {
          if (a[i] == key)
          {
               found = 1;
               break;
          }
     }
     if (found == 1)
          printf("Found");
     else
          printf("Not Found"); return 0;}
```

Today's Class (2/9/2026)

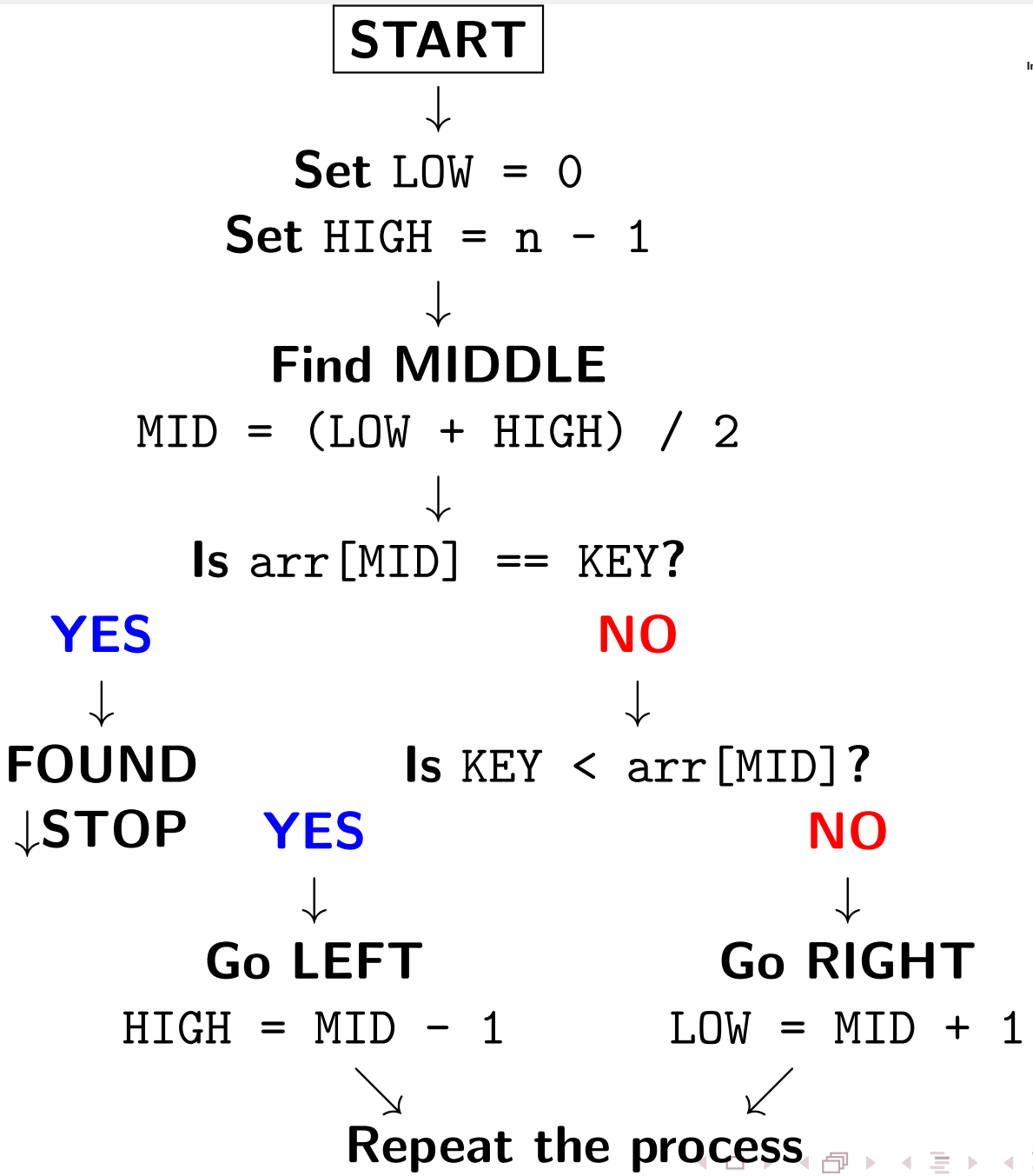
Question!!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

- Write a C program to store 7 numbers in an array in ascending order. Ask the user for a number to search and check whether that number is present in the array using binary search.
  - If the number is found, print **"Found"**.
  - Otherwise, print **"Not Found"**.
- **Example:** Suppose our array contains: 10 20 30 40 50 60 70
  And we want to search for: 60
- **Output**: **Found**

Binary Search – Algorithm

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad



Binary Search!!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

```
int main()
{   int a[7] = {10, 20, 30, 40, 50, 60, 70};
    int key, low = 0, high = 6, mid, found = 0;
    printf("Enter search key:");
    scanf("%d",&key);
    while (low <= high)
    {   mid = (low + high) / 2;
        if (a[mid] == key)
            found = 1; break;
        else if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    if (found == 1)
        printf("Found");
    else
        printf("Not Found");    return 0;}
```

From 1D Array to 2D Array

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

We have already seen a one-dimensional array:

10	20	30	40	50

```
int arr[5];
```

What if we want rows and columns?

10	20	30	40	50
60	70	80	90	100

This is a 2-dimensional array!

Multidimensional Arrays

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Declaration

```
int arr[2][5];
```

2 rows

$\times$

5 columns

$$2 \times 5 = 10$$

elements

Think of it as a table:

Row 0				
Row 1				

2 rows and 5 columns

Remember

`arr[2][5]` means **2 rows** and **5 columns**.

Two Indices = Row + Column

	Column 0	Column 1	Column 2	Column 3	Column 4
Row 0	arr[0][0]	arr[0][1]	arr[0][2]	arr[0][3]	arr[0][4]
Row 1	arr[1][0]	arr[1][1]	arr[1][2]	arr[1][3]	arr[1][4]

First index = Row **Second index = Column**

Names of the Boxes

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

For: `int arr[2][5];`

`arr[0][0]`	`arr[0][1]`	`arr[0][2]`	`arr[0][3]`	`arr[0][4]`
`arr[1][0]`	`arr[1][1]`	`arr[1][2]`	`arr[1][3]`	`arr[1][4]`

Notice the Pattern

First row:

`arr[0][0], arr[0][1], arr[0][2], arr[0][3], arr[0][4]`

Second row:

`arr[1][0], arr[1][1], arr[1][2], arr[1][3], arr[1][4]`

Every box has TWO indices.

Accessing an Element

Suppose:

10	20	30	40	50
60	70	80	90	100

To access the value in row 1, column 3: `arr[1][3]`

`arr[1][3]`

$\Downarrow$

Row 1, Column 3

Therefore:

```
arr[1][3] = 90
```

Writing into a 2D Array

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Write values

```
arr[0][0] = 10;
arr[0][1] = 20;
arr[1][0] = 30;
arr[1][1] = 40;
```

Each statement writes into one specific box.

	Col 0	Col 1
Row 0	10	20
Row 1	30	40

`arr[1][0] = 30`

⇓

Row 1, Column 0

Key Idea

Two indices tell us exactly which box we want.

Reading from a 2D Array

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Suppose:

10	20	30	40	50
60	70	80	90	100

What will this print? `printf("%d", arr[0][3]);`

Think!

Row 0, Column 3

40

Therefore:

```
arr[0][3] = 40
```

1D Array vs 2D Array

	1D Array	**2D Array**
Declaration	`int a[5];`	`int a[2][5];`
Indices	`a[3]`	`a[1][3]`
Number of indices	**1**	**2**
Think of it as	**A row**	**A table**

1D:

$a[3]$

One index identifies one box.

2D:

$a[1][3]$

Two indices identify one box: Row + Column.

Address Calculation of a 2D Array

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Consider:

```
int arr[2][5];
```

The array has 2 rows and 5 columns.

`arr[0][0]`	`arr[0][1]`	`arr[0][2]`	`arr[0][3]`	`arr[0][4]`
`arr[1][0]`	`arr[1][1]`	`arr[1][2]`	`arr[1][3]`	`arr[1][4]`

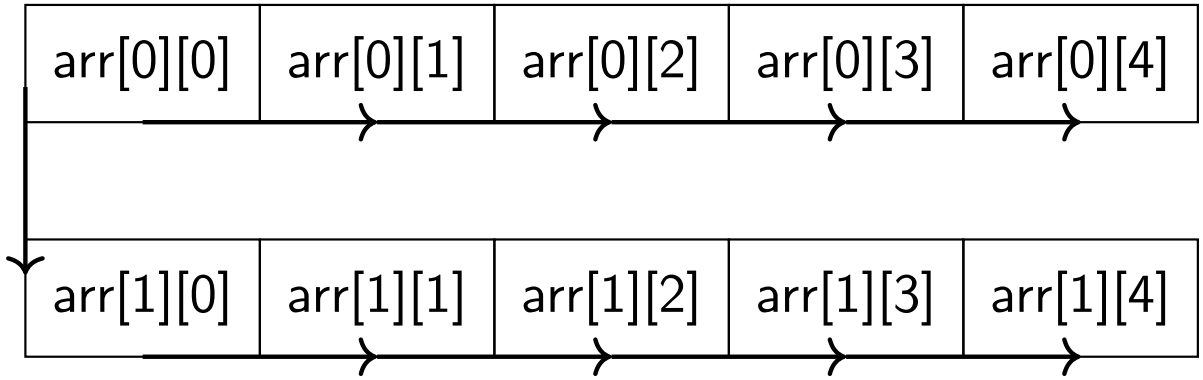
Important

The elements are stored in memory
row by row.

How Are 2D Array Elements Stored?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

C stores the elements row by row.



First complete Row 0

↓

Then complete Row 1

Remember

Row Major = Row by Row

Finding the Address of an Element

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Suppose we want:

`arr[1][3]`

How does the computer find this particular element?

[0][0]	[0][1]	[0][2]	[0][3]	[0][4]

[1][0]	[1][1]	[1][2]	[1][3]	[1][4]



We need to know how many elements come before it.

Key Idea

Address = Starting Address + Offset

Finding the Offset

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

We want:

`arr[1][3]`

Before reaching it, the computer passes:

[0][0]	[0][1]	[0][2]	[0][3]	[0][4]

[1][0]	[1][1]	[1][2]	[1][3]

5 elements from Row 0

$+$

3 elements from Row 1

$$\text{Offset} = 5 + 3 = 8 \text{ elements}$$

From Element Offset to Byte Offset

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Suppose: `int arr[2][5];` Assume:

$$\boxed{\texttt{sizeof(int)} = 4 \text{ bytes}}$$

For:

$$\boxed{\texttt{arr[1][3]}}$$

we found:

$$\text{Element offset} = 8$$

Therefore:

$$\text{Byte offset} = 8 \times 4 = 32 \text{ bytes}$$

Key Idea

8 elements away $\Rightarrow$ 32 bytes away

Address Formula for a 2D Array

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

`arr[rows][columns]`

the address of:

`arr[i][j]`

is:

$$\boxed{\text{Address}(arr[i][j]) = BA + (i \times C + j) \times S}$$

BA = Base Address
C = Number of Columns
S = Size of each Element

For : `int arr[2][5]`

$$\boxed{\text{Address of } arr[i][j] = BA + (i \times 5 + j) \times 4}$$

Number of columns is important!
It is **5** in `arr[2][5]`.

Example: Calculate the Address

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

- Consider: `int arr[2][5];`, Base Address = 1000 and: `sizeof(int)` = 4 bytes
- Find the address of: `arr[1][3]`

Example: Calculate the Address

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

- Consider: `int arr[2][5];`, Base Address = 1000 and: `sizeof(int)` = 4 bytes
- Find the address of: `arr[1][3]`

$$\begin{aligned} \text{Address} &= 1000 + (1 \times 5 + 3) \times 4 \\ &= 1000 + (5 + 3) \times 4 \\ &= 1000 + 8 \times 4 \\ &= 1000 + 32 \\ &= \boxed{1032} \end{aligned}$$

Answer

`Address of arr[1][3] = 1032`

Example 2: Calculate the Address

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

- Consider: `int arr[3][4];`, Base Address = 2000 and: `sizeof(int)` = 4 bytes
- Find the address of: `arr[2][1]`

Example 2: Calculate the Address

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

- Consider: `int arr[3][4];`, Base Address = 2000 and: `sizeof(int)` = 4 bytes
- Find the address of: `arr[2][1]`

$$2000 + (2 \times 4 + 1) \times 4 = 2000 + 9 \times 4 = 2036$$

2D Array: Row Major vs Column Major

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Row Major

Store elements **row by row**.

A_{00}	A_{01}	A_{02}
A_{10}	A_{11}	A_{12}

$$A_{00} \rightarrow A_{01} \rightarrow A_{02}$$

$$\rightarrow A_{10} \rightarrow A_{11} \rightarrow A_{12}$$

Column Major

Store elements **column by column**.

A_{00}	A_{01}	A_{02}
A_{10}	A_{11}	A_{12}

$$A_{00} \rightarrow A_{10} \rightarrow A_{01}$$

$$\rightarrow A_{11} \rightarrow A_{02} \rightarrow A_{12}$$

Row Major: Row $\rightarrow$ Row

Column Major: Column $\rightarrow$ Column

Finding the Offset in Column Major

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Suppose we want:

`arr[1][2]`

For a 2×3 array:

`arr[0][0]`	`arr[0][1]`	`arr[0][2]`
`arr[1][0]`	`arr[1][1]`	`arr[1][2]`

Column-major storage:

`arr[0][0]` $\rightarrow$ `arr[1][0]` $\rightarrow$ `arr[0][1]` $\rightarrow$ `arr[1][1]` $\rightarrow$ `arr[0][2]`

$\rightarrow$ `arr[1][2]` Before `arr[1][2]`:

4 elements

Calculating the Element Offset

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

We want:

`arr[1][2]`

There are **2 rows** in each column.

We need to skip:

$$\underbrace{2}_{\text{rows per column}} \times \underbrace{2}_{\text{columns before column 2}}$$

Then move to:

$$\underbrace{1}_{\text{row 1}}$$

Therefore:

$$\boxed{\text{Element Offset} = (2 \times 2) + 1 = 5}$$

Complete columns to skip + row to move

Column-Major Address Formula

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

For:

`arr[rows][columns]`

the address of:

`arr[i][j]`

is:

$$\boxed{\text{Address}(arr[i][j]) = BA + (j \times R + i) \times S}$$

where:

BA = Base Address
R = Number of Rows
S = Size of each Element

Remember

Column Major: multiply by NUMBER OF ROWS

Example: Column-Major Address

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Consider:

```
int arr[2][3];
```

$BA = 1000$ `sizeof(int)` $= 4$

Find the address of:

`arr[1][2]`

Example: Column-Major Address

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Consider:

`int arr[2][3];`

$BA = 1000$ `sizeof(int)` $= 4$

Find the address of:

`arr[1][2]`

Using:

$$\text{Address} = BA + (j \times R + i) \times S$$

$$= 1000 + (2 \times 2 + 1) \times 4$$
$$= 1000 + (4 + 1) \times 4$$
$$= 1000 + 20$$
$$= \boxed{1020}$$

`Address of arr[1][2] = 1020`

Today's Class (4/9/2026)

ASSIGNMENT

Digital Painting: Teachers Day with Christmas Tree using For Loops

Write a program in C/C++ that creates a digital painting using for loops.
The painting should include:

1. A **Christmas Tree** made using the **'*'** character.
2. Decorations on the tree using **'o'** (balls).
3. A **Star** on the top of the tree using **'*'**.
4. The message **"HAPPY TEACHERS DAY"**
5. A Trunk for the tree using '|' or '#'.
6. Use only **for loops** for drawing all parts (tree, star, decorations, trunk and text).

Requirements:

- Use for loops only. (No while, do-while or recursion)
- Use nested for loops wherever necessary.
- Output should be aligned properly.
- The size of the tree should be at least 9 levels.

Sample Output: →



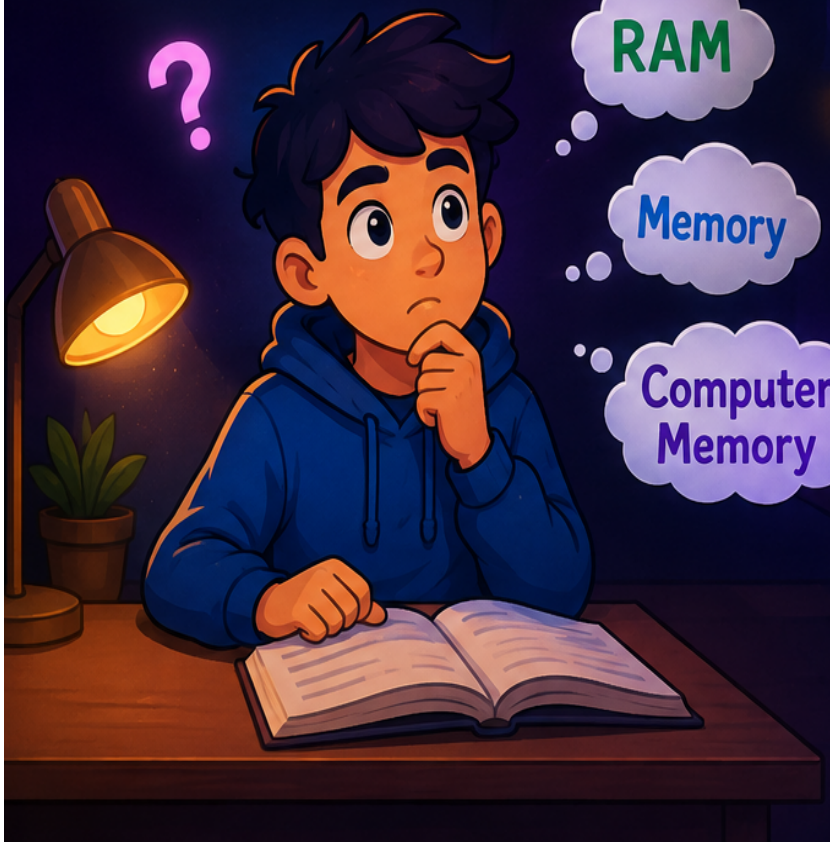
Where does a variable live?

Suppose I create a variable called **marks**.
Where is **marks** actually stored?

RAM

Memory

Computer Memory



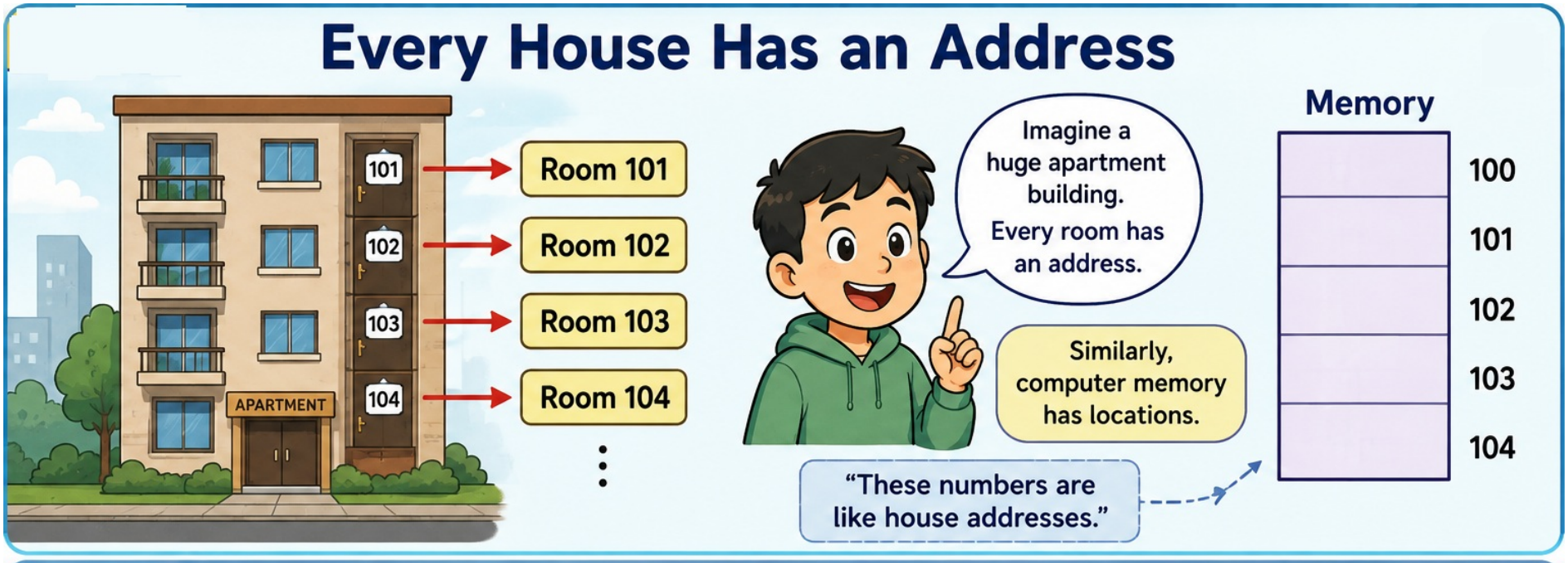


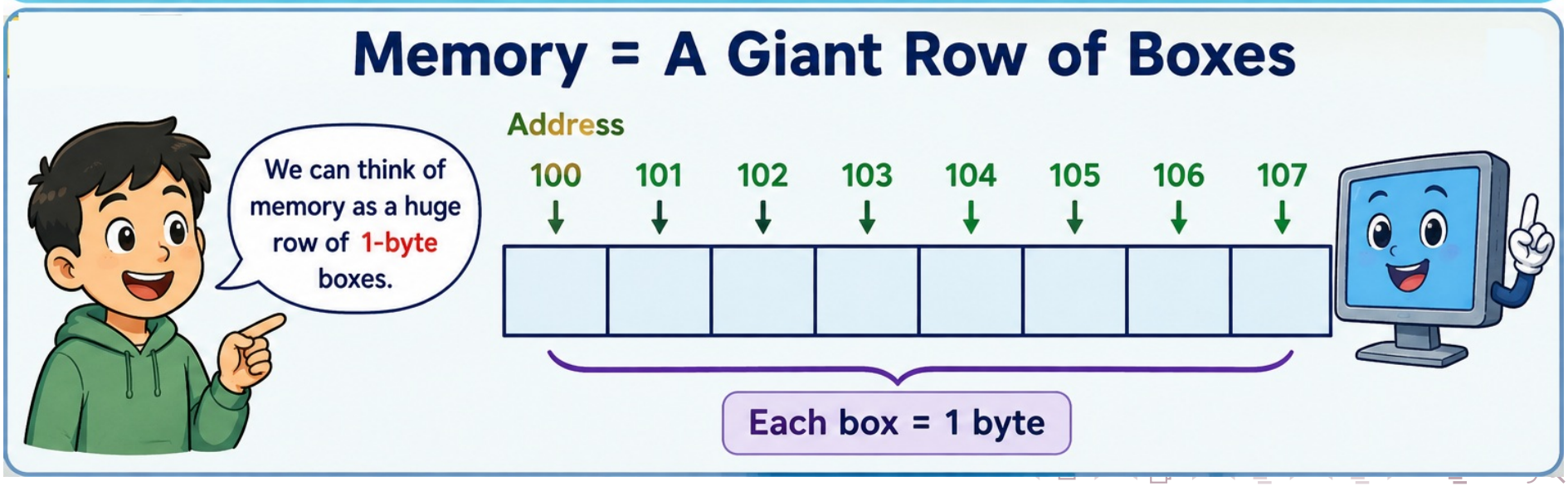
Okay. But **how** does the computer find it?

?

Every House Has an Address

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad





Different Variables Need Different Space

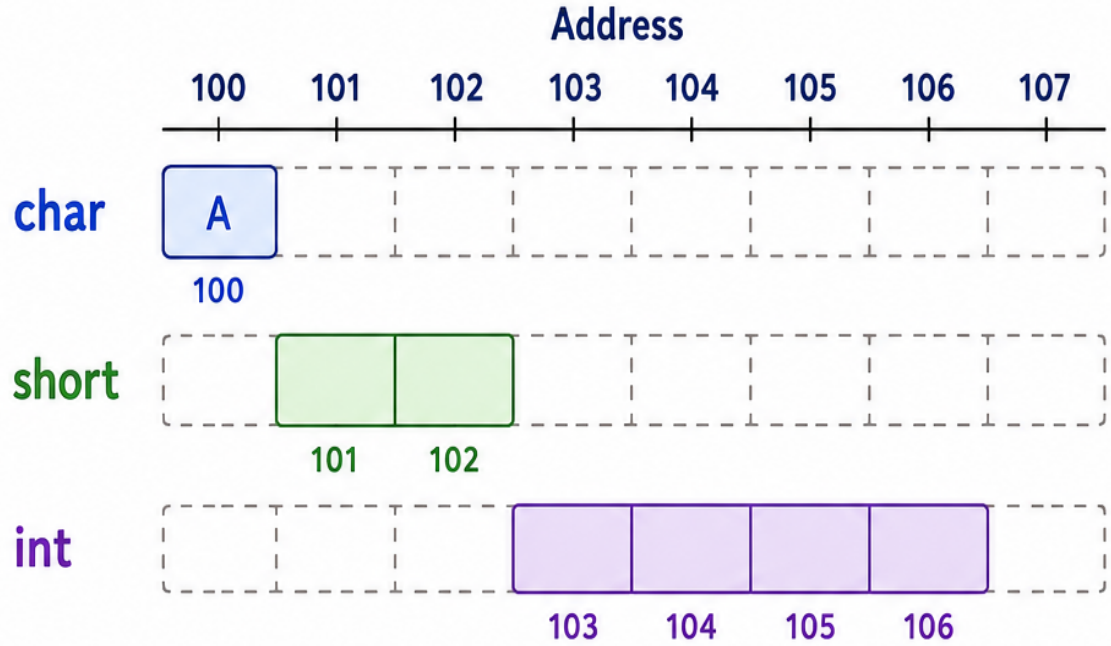


Does every variable need the same amount of memory? → **No.**

char → 1 byte

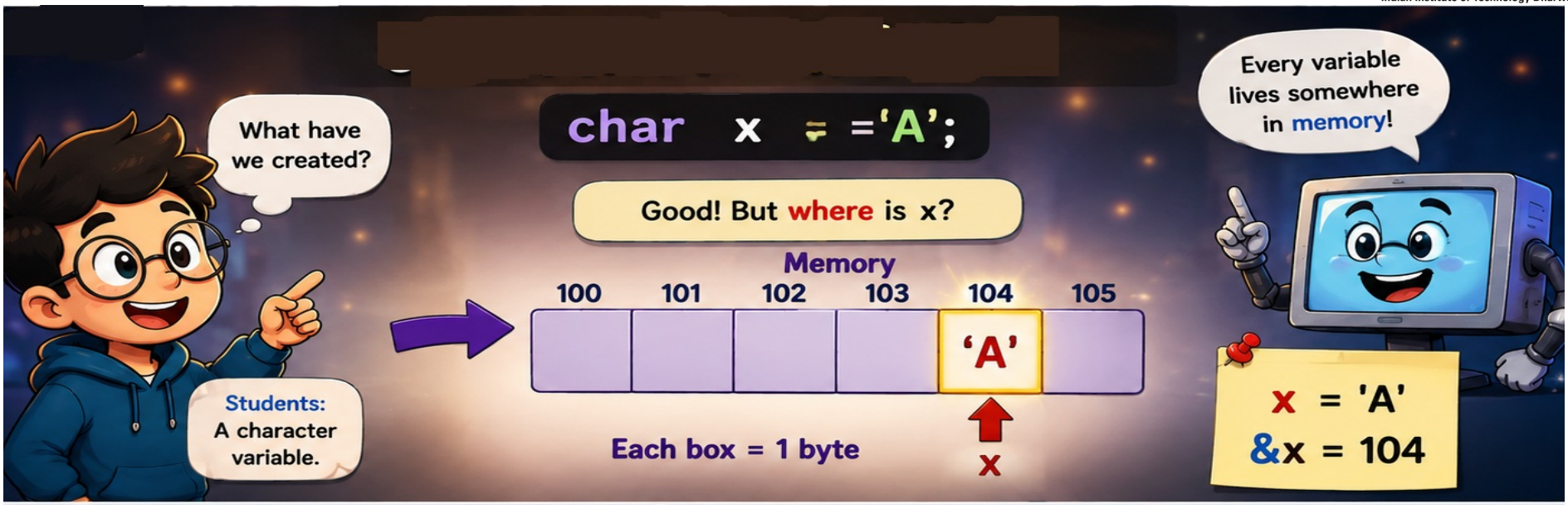
short → 2 bytes

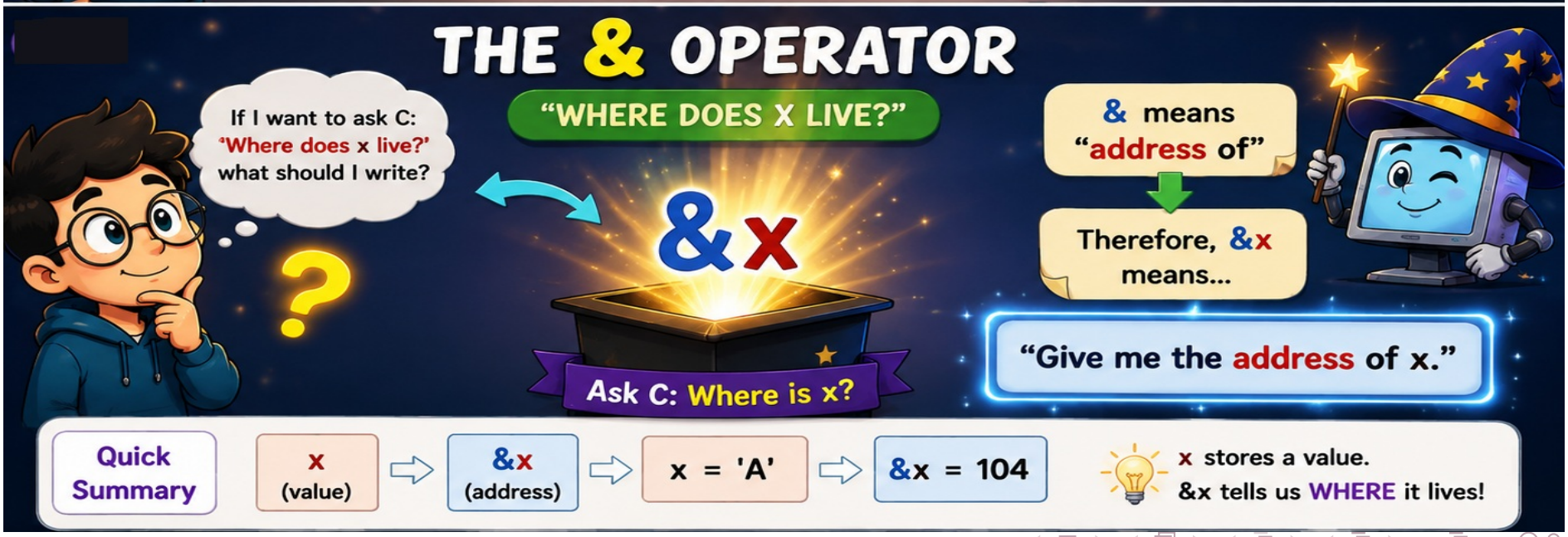
int → 4 bytes



The exact sizes depend on the system, but for our examples we will use the common sizes: **char** = 1 byte, **short** = 2 bytes, **int** = 4 bytes.

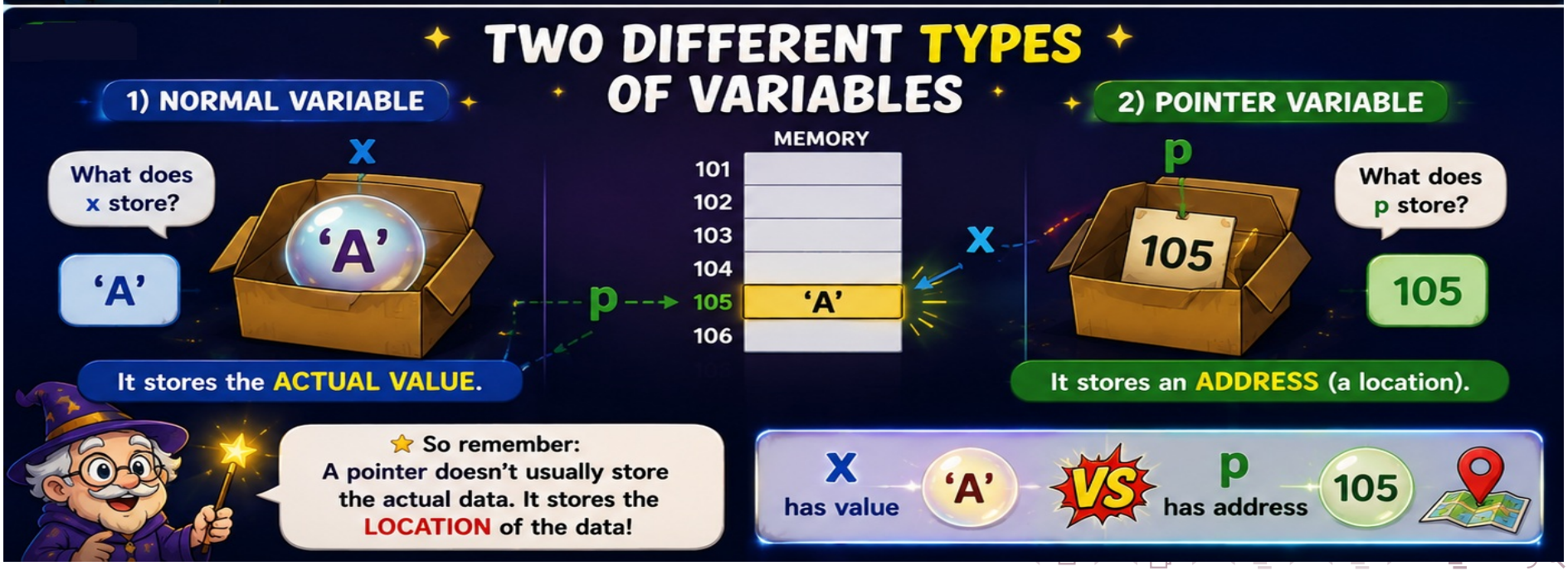
Meet Our x


What have we created?
char x = 'A';
Every variable lives somewhere in memory!
Good! But where is x?
Memory
100 101 102 103 104 105
'A'
x
Each box = 1 byte
Students: A character variable.
x = 'A'
&x = 104


THE & OPERATOR
"WHERE DOES X LIVE?"
If I want to ask C: 'Where does x live?' what should I write?
?
&x
Ask C: Where is x?
& means "address of"
Therefore, &x means...
"Give me the address of x."
Quick Summary
x (value)
&x (address)
x = 'A'
&x = 104
x stores a value.
&x tells us WHERE it lives!

Pointers

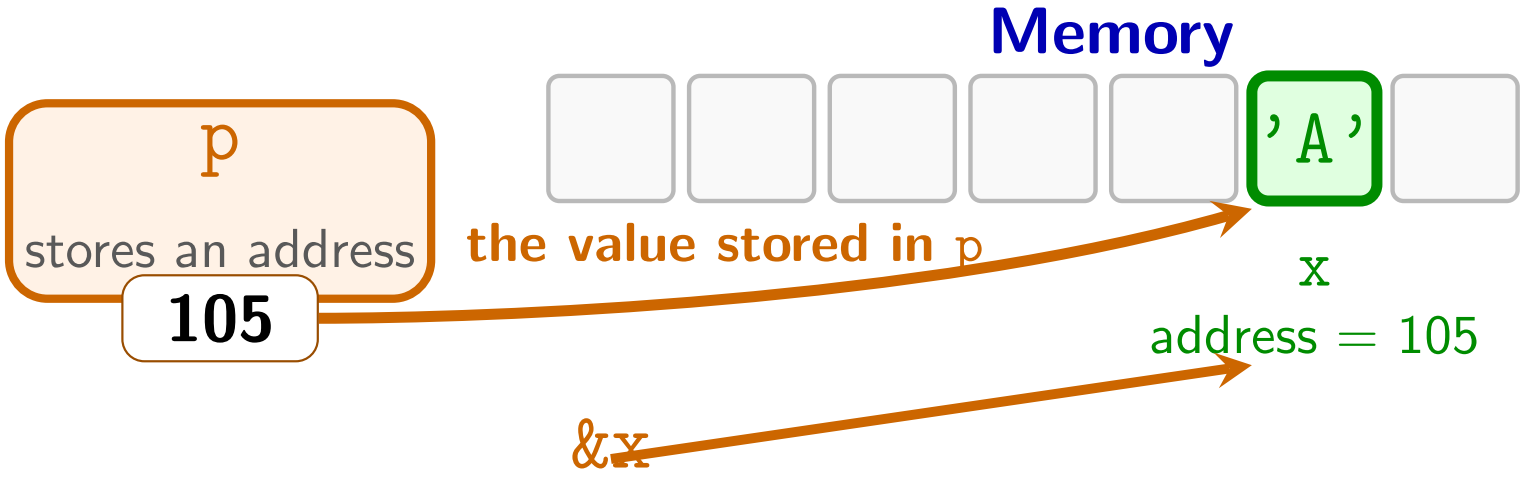

WE HAVE A PROBLEM!
Suppose I know the address of x.
Can I keep that address somewhere?
Of course!
Introduce:
char *p;
Say:
"p is a pointer variable."
But don't stop there!
Explain:
p
stores an address
So:
NORMAL VARIABLE
x → stores a value
POINTER VARIABLE
p → stores an address


TWO DIFFERENT TYPES OF VARIABLES
1) NORMAL VARIABLE
What does x store?
'A'
x
'A'
It stores the ACTUAL VALUE.
MEMORY
101
102
103
104
105
106
'A'
p
x
2) POINTER VARIABLE
What does p store?
105
p
105
It stores an ADDRESS (a location).
So remember:
A pointer doesn't usually store the actual data. It stores the LOCATION of the data!
x has value 'A'
VS
p has address 105

Giving p the Address of x

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

```
char x = 'A';
char *p;
p = &x;
```

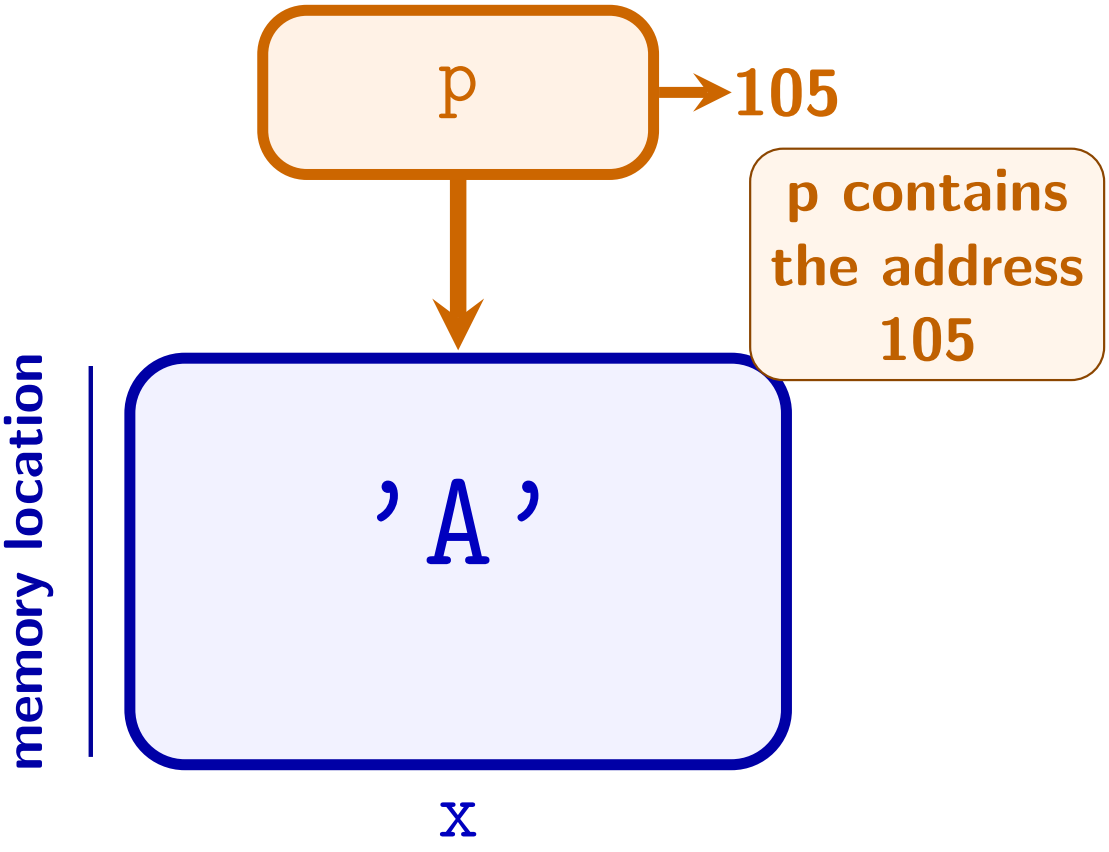


`p = &x;`

"Put the **address of** x inside p."

The Most Important Pointer Diagram

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad



Address: 105

What does p contain?
105

p **does not contain** 'A'.

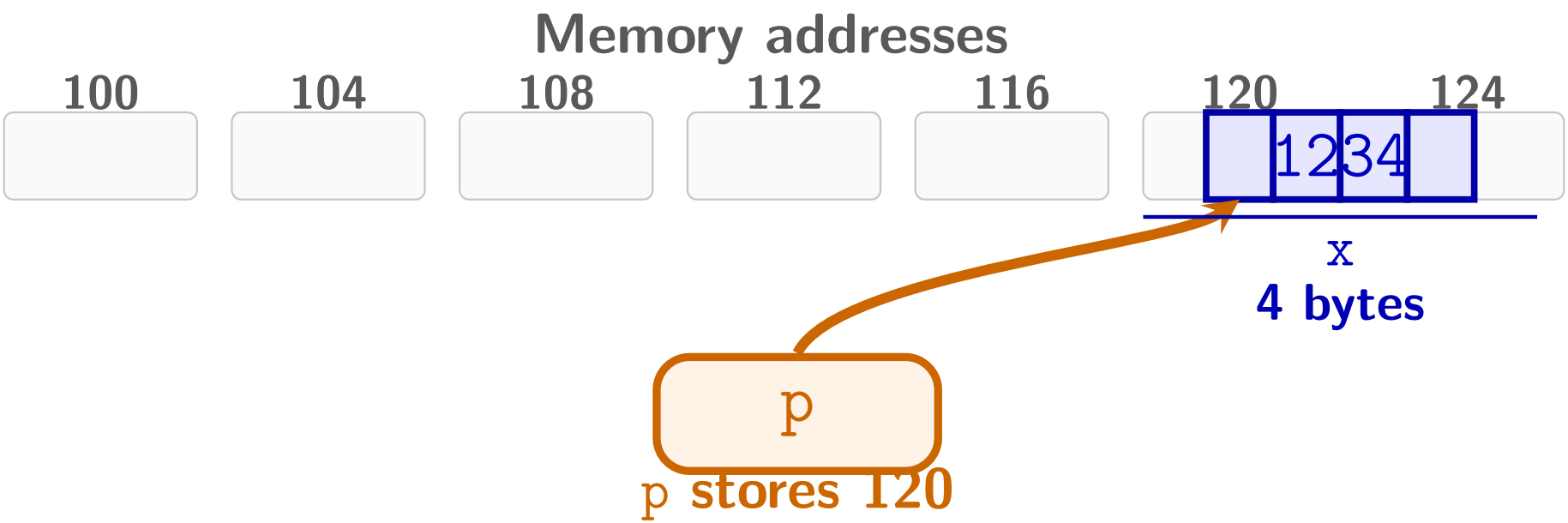
What does x contain?
'A'

p **contains the address of** x.

Why `int *p`?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
dian Institute of Technology Dharwad

```
int x = 1234;
int *p;
p = &x;
```

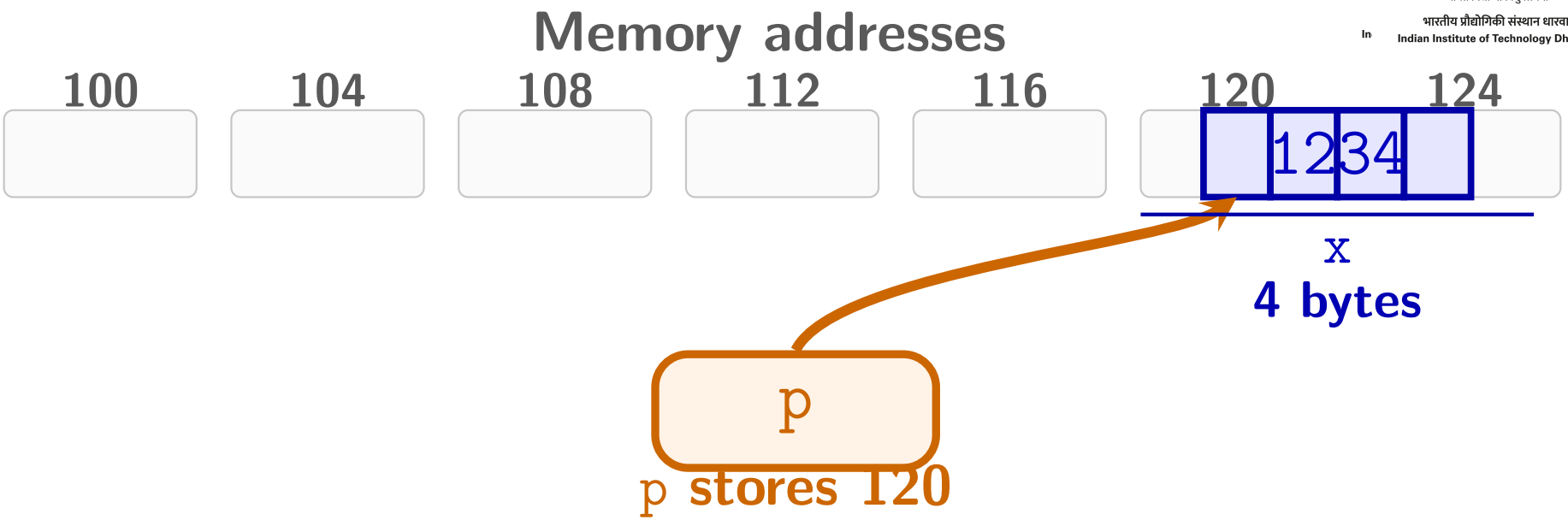


If `x` starts at address **120**, then an `int` occupies:

120 121 122 123

The next `int`-sized location begins at **124**.

Why `int *p`?



If `x` starts at address **120**, then an `int` occupies:

120 121 122 123

The next `int`-sized location begins at **124**.

The pointer's type tells the compiler what kind of object it points to.

Why Not Just *p?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

"Why do we write `int *p` instead of simply `*p`?"

`int *p` —different type→ `char *p`

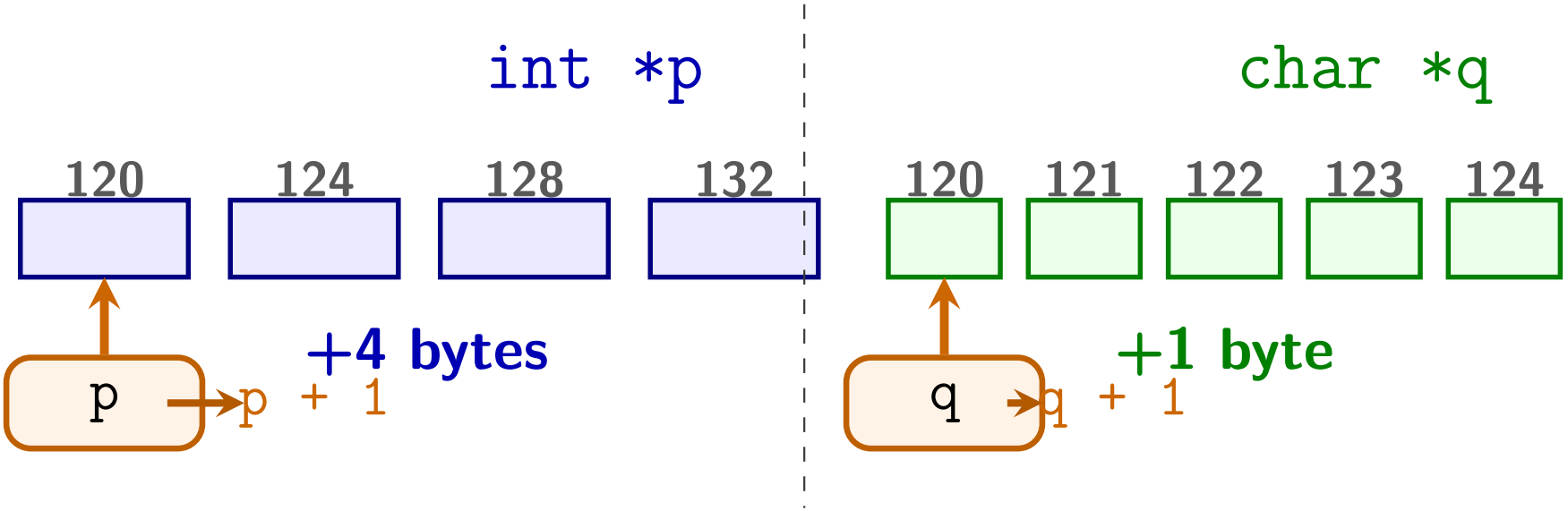
pointer to an `int` | **pointer to a `char`**

The declaration tells C:

"What kind of object does `p` point to?"

The Type Also Controls Pointer Arithmetic

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

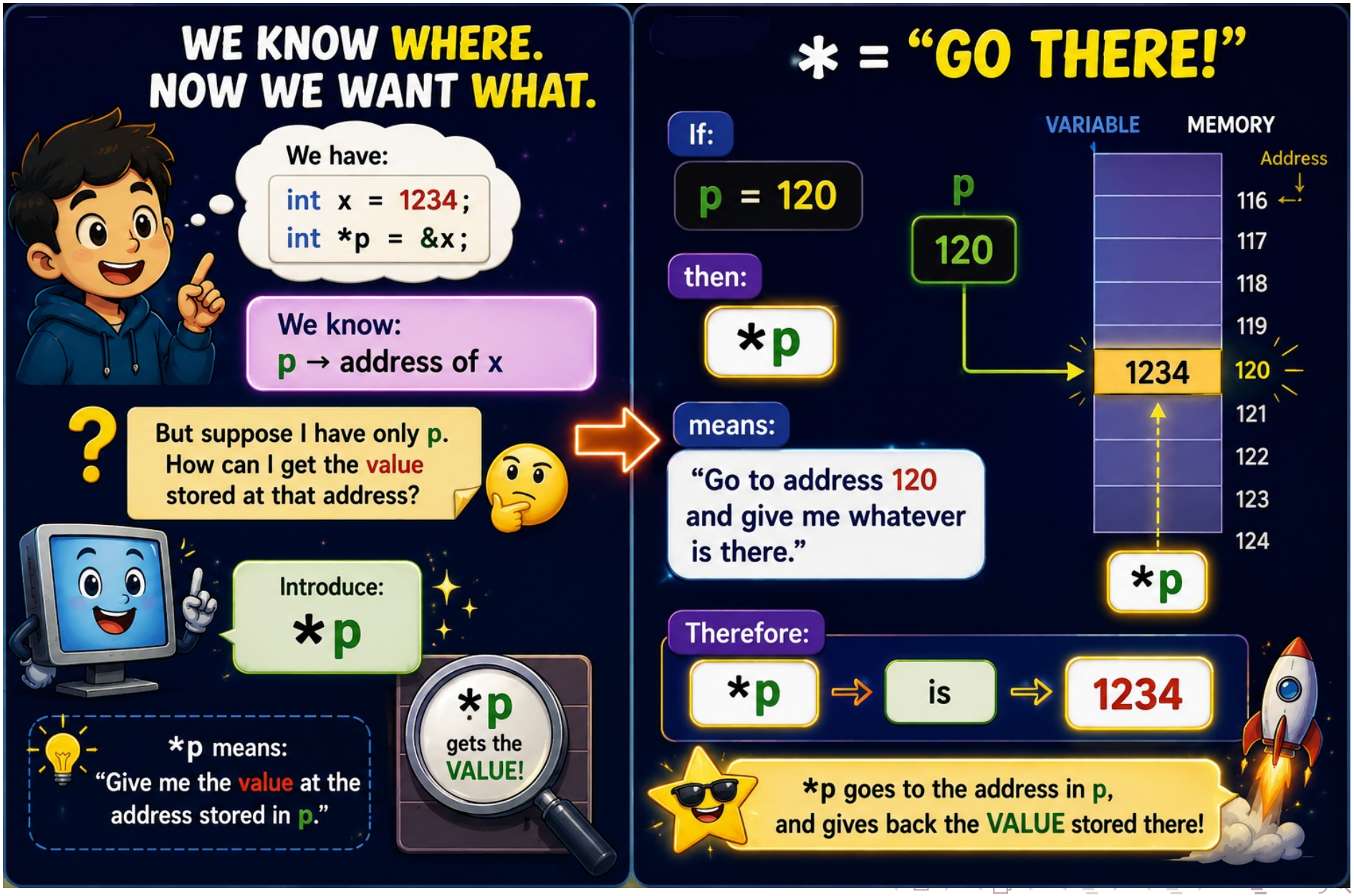


p + 1 does **not necessarily mean "next byte".**

It means: **"move to the next object of the pointed-to type."**

* = “Go There!”

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad



& and *: What Do They Mean?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

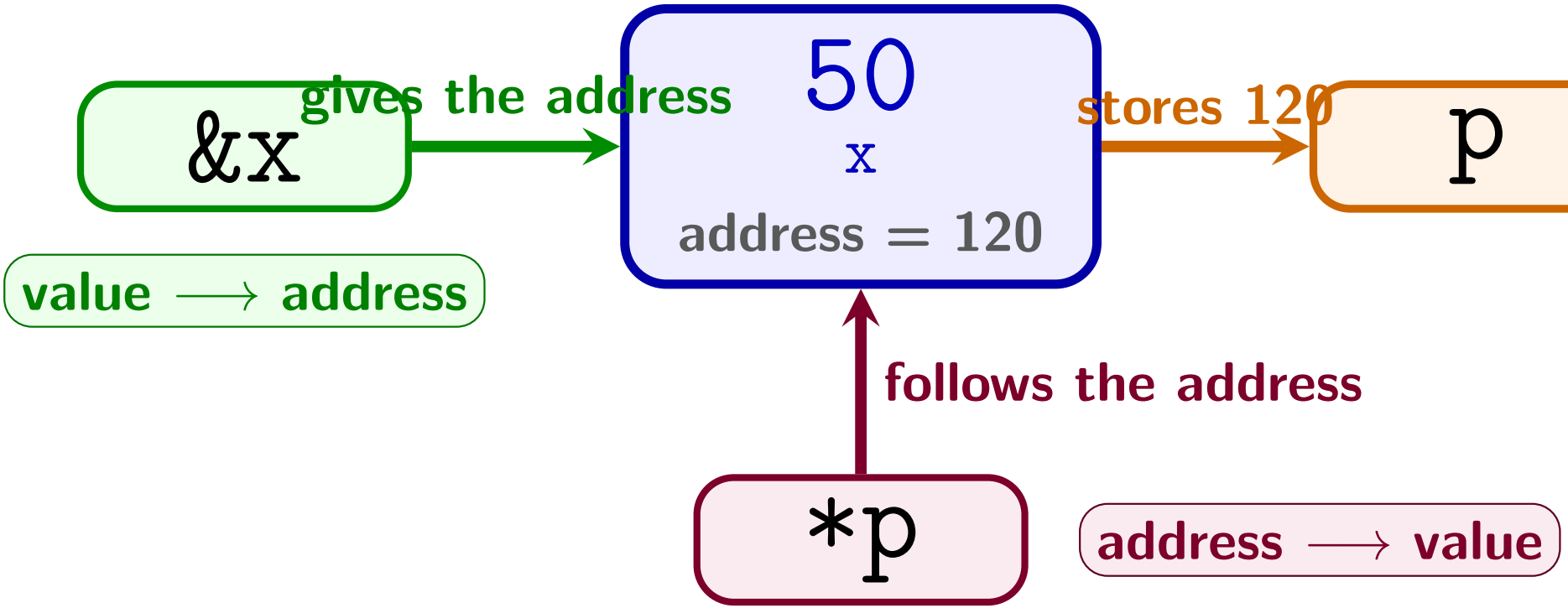
```
int x = 50;
int *p = &x;
```

x = ? &x = ? p = ? *p = ?

"Don't calculate yet. What do you THINK each one represents?"

Follow the Value and the Address

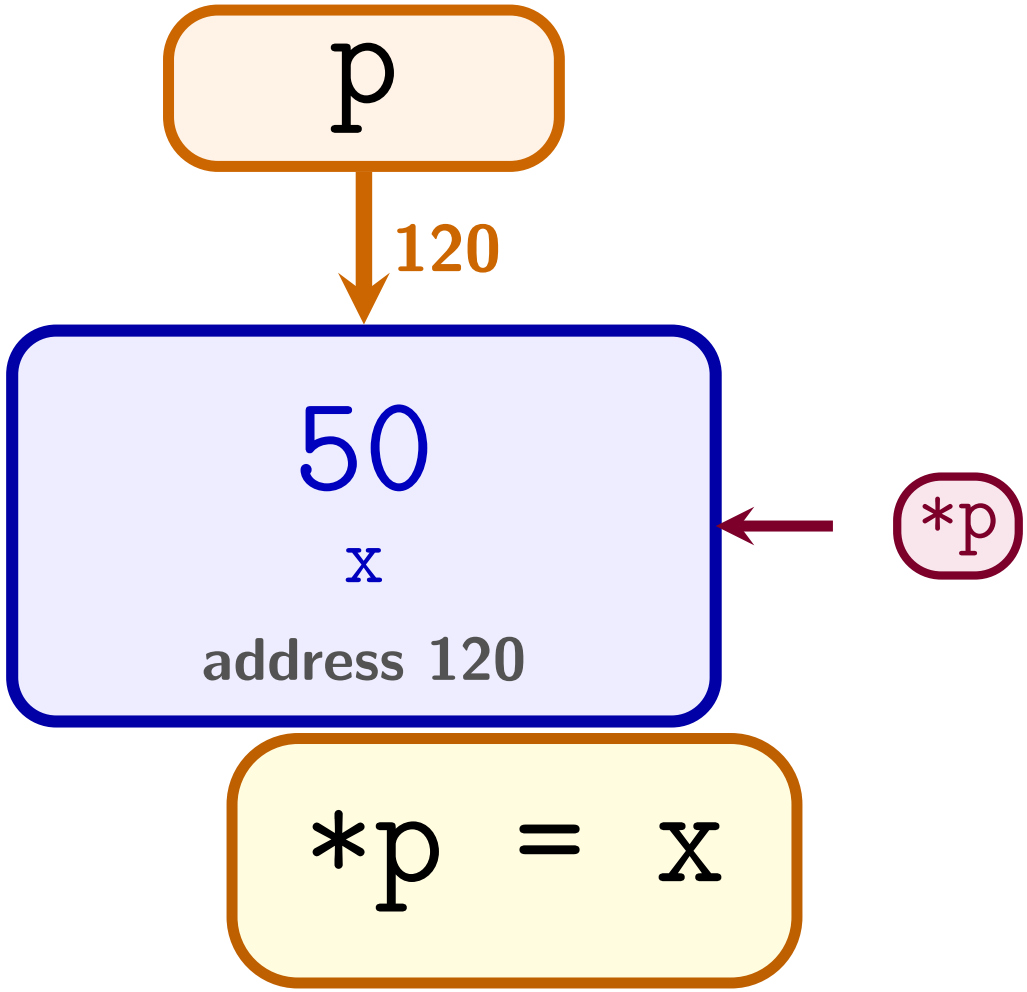
॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad




```
&x = 120        p = 120        *p = 50
```

The Beautiful Observation

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad



Because `p` contains the address of `x`,

`*p` means: "go to that address and get the value."

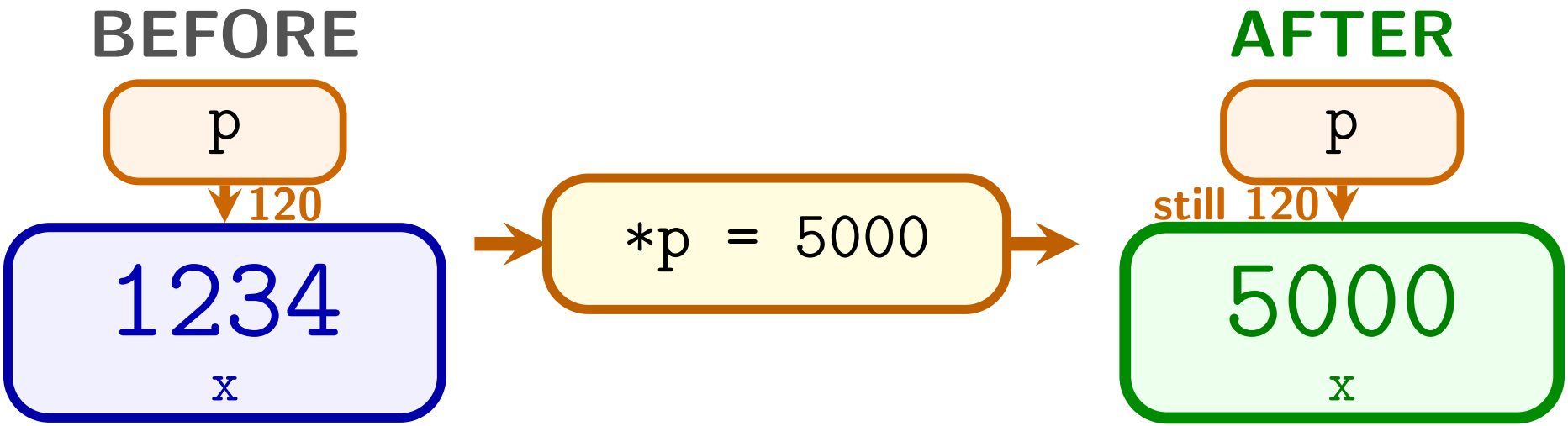
The Magic Trick


```
int x = 1234;
int *p = &x;
*p = 5000;
```

The Magic Trick

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

```
int x = 1234;
int *p = &x;
*p = 5000;
```



*p = 5000 means:

"Go to the address stored in p and change the value there."

Trap Question!!

Predict the Output

```
int x = 10;
int *p = &x;

*p = 25;

printf("%d", x);
```

What will be printed?

Trap Question!!

Predict the Output

```
int x = 10;
int *p = &x;

*p = 25;

printf("%d", x);
```

What will be printed?

Answer: 25

Because `*p` refers to the value stored at the address of `x`.

Pointer as a Remote Control

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

Remember

```
int x = 10;
int *p = &x;
```

p $\longrightarrow$ address of x *p $\longrightarrow$ **value of** x

Think of p as a remote control!

$\boxed{*p}$

"Go to the address stored in p and access the value there."

So we can change x through the pointer

```
*p = *p + 10;
```

is equivalent to

```
x = x + 10;
```

$\boxed{x = 20}$

Challenge: What Does ++*p Mean?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Given

```
int x = 10;
int *p = &x;

++*p;
```

What is the value of `x` now?

Challenge: What Does ++*p Mean?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Given

```
int x = 10;
int *p = &x;

++*p;
```

What is the value of `x` now?

Answer: 11

`++*p` means `++(*p)`

Why?

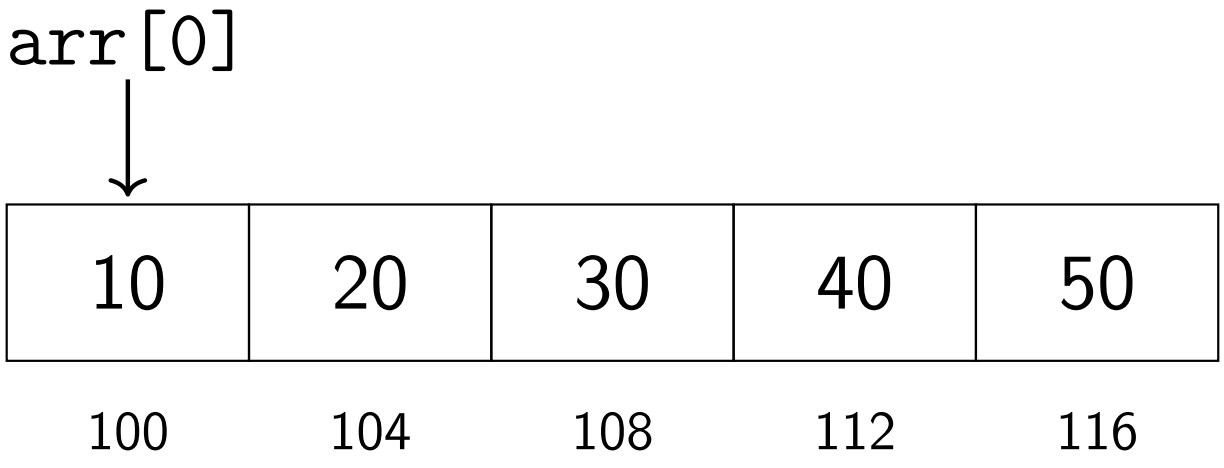
The prefix ++ operates on the value obtained by dereferencing `p`.

Remember Our Array?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

We already know this!

```
int arr[5] = {10, 20, 30, 40, 50};
```



Think!

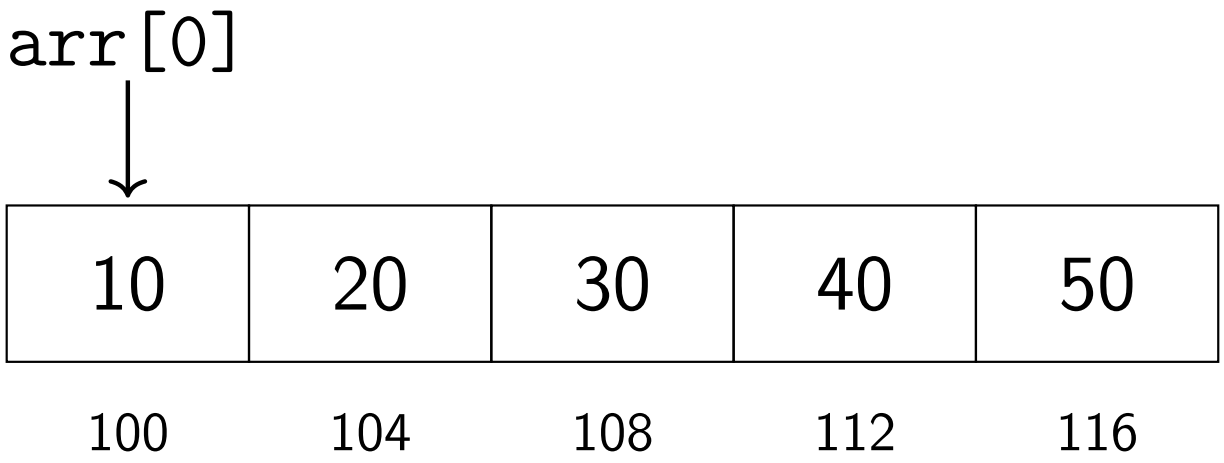
Each element has a value **and** an address.

Remember Our Array?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

We already know this!

```
int arr[5] = {10, 20, 30, 40, 50};
```



Think!

Each element has a value **and** an address.

Question

What if we want a pointer to one particular element?

Point to Any Element



Given

```
int arr[5] = {10, 20, 30, 40, 50};

int *p = &arr[3];
```

Where is `arr[3]`?

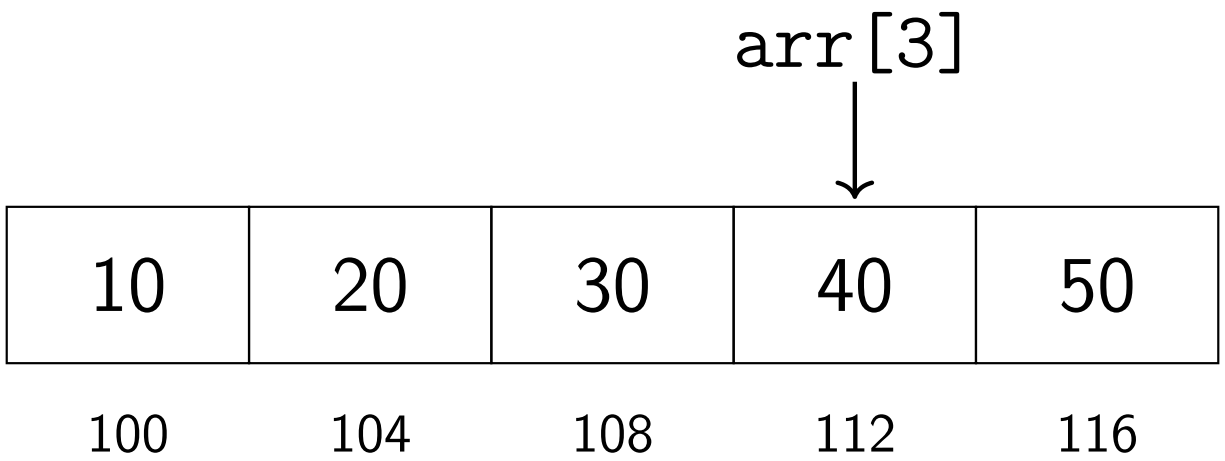
Point to Any Element

Given

```
int arr[5] = {10, 20, 30, 40, 50};

int *p = &arr[3];
```

Where is `arr[3]`?



`arr[3] is at address 112`

Therefore:

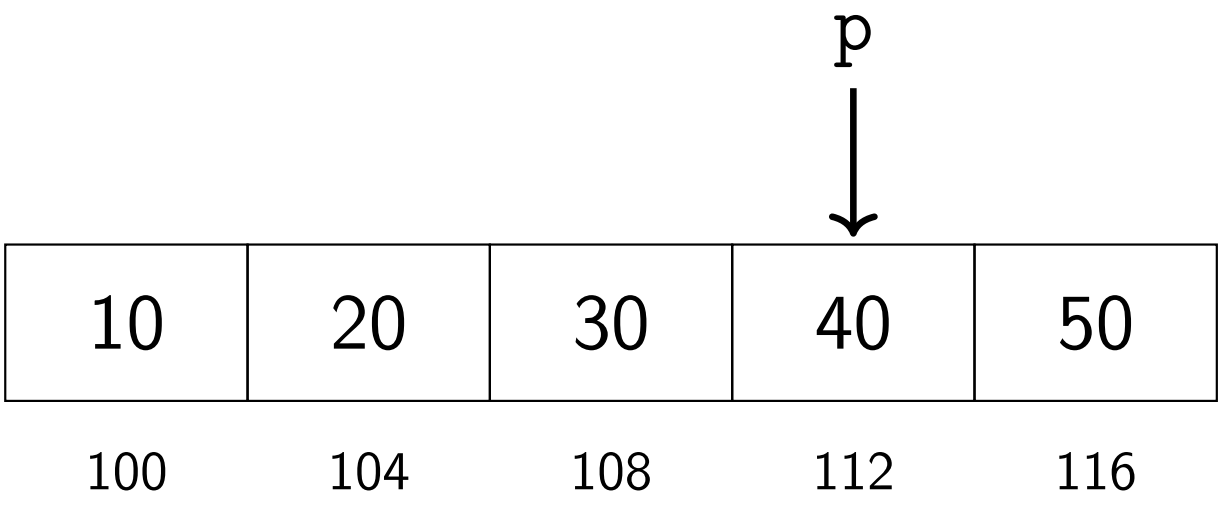
$$p = 112$$

Follow the Pointer!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Given:

```
int *p = &arr[3];
```

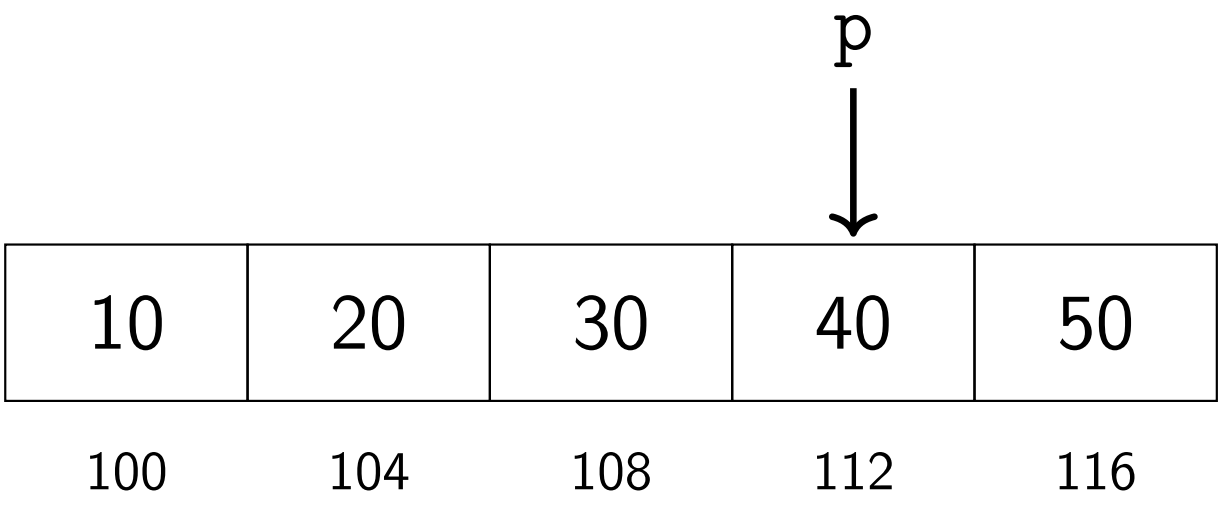


If `p = 112`, what does `*p` give us?

Follow the Pointer!

Given:

```
int *p = &arr[3];
```



If `p = 112`, what does `*p` give us?

$$*p = 40$$

Key Idea

`p` tells us **WHERE**. `*p` gives us **WHAT**.

Pointer Arithmetic: What Does `p + 1` Mean?

A common guess

"If `p = 112`, then `p + 1 = 113`?"

Pointer Arithmetic: What Does `p + 1` Mean?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

A common guess

"If `p = 112`, then `p + 1 = 113`?"

No!

Pointers don't move like ordinary numbers

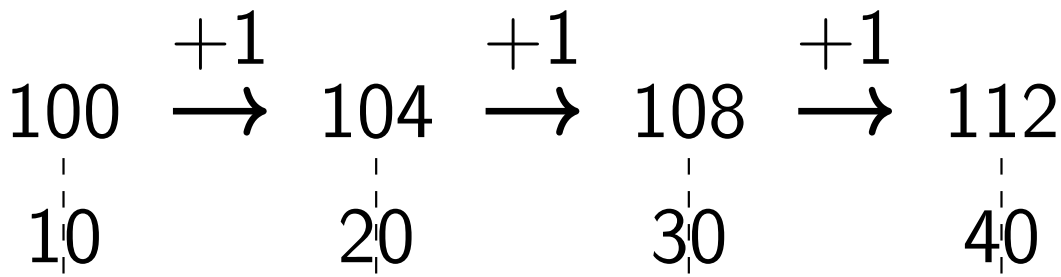
For an `int *`, `p + 1` means: **"Move to the next `int`."**

Expression	Address
p	112
p+1	116
p+2	120
p+3	124

```
int = 4 bytes
```

```
p + 1 = p + sizeof(int)
```

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad



What does `p + 1` mean?

Go to the next element of the type.

`int *p`

$+1 \Rightarrow +4$ bytes

`char *p`

$+1 \Rightarrow +1$ byte

Pointer Quiz

Given

```
int arr[] = {10, 20, 30, 40, 50};

int *p = &arr[2];
```

Fill in the blanks!

*p _____ =

Pointer Quiz

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Given

```
int arr[] = {10, 20, 30, 40, 50};

int *p = &arr[2];
```

Fill in the blanks!

`*p` = $\boxed{30}$

`*(p+1)` = $____$

Pointer Quiz

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Given

```
int arr[] = {10, 20, 30, 40, 50};

int *p = &arr[2];
```

Fill in the blanks!

`*p` = 30

`*(p+1)` = 40

`*(p+2)` =

Pointer Quiz

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Given

```
int arr[] = {10, 20, 30, 40, 50};

int *p = &arr[2];
```

Fill in the blanks!

`*p` = 30

`*(p+1)` = 40

`*(p+2)` = 50

`*(p-1)` =

Pointer Quiz

Given

```
int arr[] = {10, 20, 30, 40, 50};

int *p = &arr[2];
```

Fill in the blanks!

$$\texttt{*p} = \boxed{30}$$

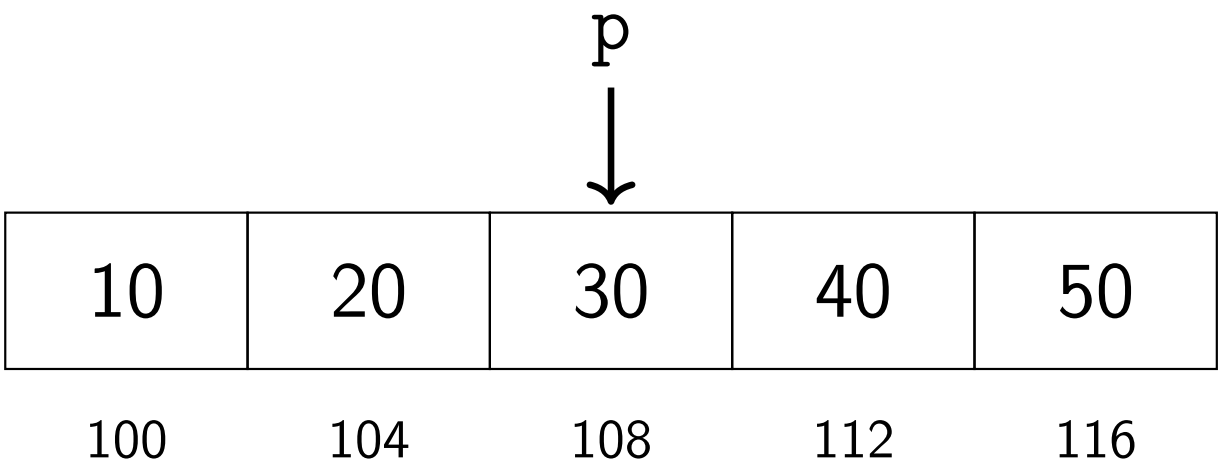
$$\texttt{*(p+1)} = \boxed{40}$$

$$\texttt{*(p+2)} = \boxed{50}$$

$$\texttt{*(p-1)} = \boxed{20}$$

Let's Walk the Pointer!

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad



Expression	Value
`*p`	30
`*(p+1)`	40
`*(p+2)`	50
`*(p-1)`	20

The Big Connection

`*(p+1)` accesses the **next element**.

The Magic Connection

Suppose:

```
int *p = &arr[2];
```

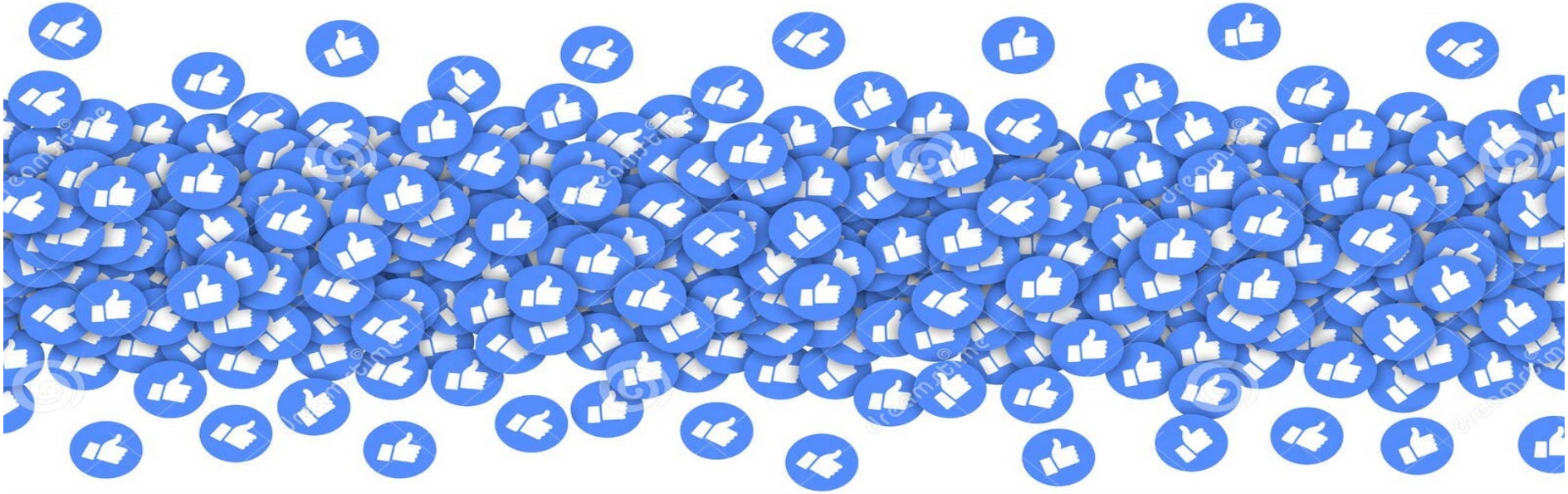
```
*p = arr[2]
```

```
*(p+1) = arr[3]
```

```
*(p+2) = arr[4]
```

Remember

Pointer arithmetic lets us walk through an array!



Thank you for your kind attention!!